ChatGPT

Feasibility of a Software Factory for LEGO-Based Self-Assembling Learning Robots

Executive summary

A LEGO-centered approach to a “learning robot that can build additional robots” is **feasible as a staged research and education prototype**, but **not as a pure LEGO-only, fully autonomous self-replicating system** with today’s official LEGO hardware and software stack. The reason is simple: LEGO’s programmable hubs are very good at **embedded control, classroom robotics, and modular mechatronics**, but they are not designed to be the sole platform for **high-precision assembly planning, dense perception, fleet orchestration, wiring, automated commissioning, or robust recovery from contact-rich insertion failures**. Those higher-level functions need to live on external compute such as a Raspberry Pi or workstation, with the LEGO hardware acting as the modular mechanical and low-level actuation layer. [1]

The direct answer to your earlier question is **yes**: LEGO kits **can take custom instructions in the form of code**, but the depth varies by platform. **SPIKE Prime, MINDSTORMS EV3, and Robot Inventor** officially support custom programming; **Powered Up** and **BOOST** are also programmable, but more through app-centric block coding and protocol-level control than through a rich, officially documented on-hub text programming model. Among official platforms, **EV3 remains the most open legacy option** because LEGO still provides MicroPython, MakeCode, developer kits, a firmware developer edition, and even firmware source code. **SPIKE Prime is the best current official educational platform**, but it is still an embedded microcontroller environment with limited memory and processing power, so serious autonomy should be offloaded. [2]

The most viable architecture is therefore a **hybrid software factory**: LEGO or LEGO-compatible building blocks for standardized mechanical modules; **SPIKE/Powered Up/EV3-class components** for reusable actuators and sensors; **Raspberry Pi Build HAT, BrickPi, or Pybricks** for broader programmability; and **OpenMV or Pi-based vision** for part localization, AprilTags, and failure detection. On top of that, a software-factory pipeline should version **CAD models, part catalogs, instruction graphs, simulation artifacts, firmware, and calibration data**, then continuously test them in simulation and on hardware. That is a much stronger path than trying to make one LEGO hub do everything. [3]

My bottom-line judgment is this: **building additional robots is realistic if you constrain the task to kitted parts, fixtures, standardized module families, and mostly passive or semi-prepared subassemblies**. It becomes far less realistic if “build additional robots” means **fully autonomous picking from loose unsorted bins, routing wires, inserting batteries, flashing firmware, calibrating sensors, and reproducing the entire build cell without human help**. Research systems have already shown that LEGO is an excellent testbed for automated assembly, digital twins, tactile supervision, and multi-robot planning, but the best results still rely on **specialized tooling, vision, force/tactile feedback, and external planning stacks**, not on educational hubs alone. [4]

Decision framing and what is actually achievable

A useful way to analyze your idea is to separate four very different ambitions.

Capability target	Practical feasibility	What it really means	Why
Programmable LEGO robot behavior	**High**	Uploading custom logic to a LEGO hub or controlling motors/sensors from an app, Python runtime, or external controller	Official support exists across SPIKE Prime, EV3, Robot Inventor, Powered Up, and BOOST, though the depth varies widely. [5]
Autonomous assembly of passive LEGO modules	**Moderate**	Picking, aligning, and snapping standardized bricks or beams into prepared fixtures or partially built structures	Research results show LEGO assembly is feasible, but it is contact-rich, precision-sensitive, and benefits from digital twins, force/tactile sensing, and tool-tip vision. [6]
Autonomous construction of daughter robots with motors/sensors/hubs from kitted parts	**Moderate to low**	Building a second robot when electronics are pre-presented, fixtures are used, and commissioning is partly automated off-board	Mechanical assembly is only one part of the problem; power, firmware, calibration, and cable management dominate late-stage complexity. [7]
Fully autonomous LEGO self-replication	**Low**	Robot builds another robot from loose parts, commissions it, and repeats the process without human intervention	Current LEGO hubs are educational, embedded, and app-centric; recent research still relies on specialized end effectors, planning stacks, and digital twins. [8]

The most important implication is that a **“software factory” model is not optional** here; it is the only sensible way to keep the concept under control. In this context, a software factory means a pipeline that turns a robot definition into **versioned artifacts**: a machine-readable part list, a CAD model, an assembly graph, simulation assets, vision/calibration assets, hub code, coprocessor code, and deployment packages, all of which are automatically tested and regenerated when a design changes. GitHub Actions and GitLab CI/CD already provide the automation layer for that kind of workflow, while BrickLink Studio, Gazebo, and Isaac Sim provide pieces of the CAD/instruction/simulation side. [9]

LEGO programmable platforms

LEGO’s current programmable ecosystem is fragmented across **current education products**, **retired consumer products**, and **accessory/control ecosystems**. That fragmentation matters a lot for a self-assembly project, because lifecycle risk is almost as important as technical capability. [10]

Platform comparison

Platform	Status in June 2026	I/O and built-ins	Connectivity	What LEGO publicly discloses about compute	Practical implication for your use case	Sources
SPIKE Prime	Sold in the U.S.; some regional LEGO Education pages mark the set as retiring June 30, 2026	6 I/O ports, 5×5 light matrix, speaker, 6-axis gyro, rechargeable battery	USB and BLE	Official docs describe the hub as a microcontroller with limited memory and processing power, without prominently publishing CPU/RAM figures	Best official starting point if you want current-ish LEGO-native education tooling, but not enough compute for perception/planning	11
MINDSTORMS EV3	Retired; LEGO states classroom EV3 app downloads remain on LEGO Education until July 2026	4 motor ports, 4 sensor ports, USB, microSD, display, buttons; official product page names an ARM9 processor	USB, Bluetooth, Wi-Fi via USB dongle	ARM9 is explicitly disclosed; microSD path is officially documented	The strongest official legacy option if openness and low-level control matter more than staying on LEGO's newest stack	12

Platform	Status in June 2026	I/O and built-ins	Connectivity	What LEGO publicly discloses about compute	Practical implication for your use case	Sources
Robot Inventor	Retired in 2023; LEGO says the hub can still be programmed on the SPIKE app	6 I/O ports, 5×5 LED matrix, speaker, 6-axis gyro, rechargeable battery; set includes 4 medium motors, color and distance sensors	Micro USB and Bluetooth	LEGO does not prominently publish CPU/RAM; official docs emphasize app-based coding	Useful if you already own it, but a poor base for new long-term platform investment	[13]
Powered Up Hub / Technic Hub	Current accessory/control ecosystem	Basic Hub: 2 I/O ports; Technic Hub: 4 I/O ports plus tilt sensor	Bluetooth	No prominent official CPU/RAM disclosure; official focus is app/protocol control	Excellent source of reusable motors, hubs, and sensors; weak as a standalone autonomy brain	[14]
BOOST Move Hub	Retired; LEGO says interactive features now live in the Powered Up app	2 ports, 2 integrated position motors, integrated tilt sensor	Bluetooth	No prominent official CPU/RAM disclosure	Fine as a legacy motion module, but not a durable platform choice for a serious build-cell roadmap	[15]

Official SDKs, APIs, and custom instruction depth

The “SDK” story is uneven. In current LEGO products, many of the most important APIs live inside apps and online help rather than as clean standalone developer packages. EV3 is the exception: it still has genuine developer kits and firmware materials. [16]

Platform	Official software surfaces	Official languages	Can you upload custom code?	Low-level motor/sensor control	Official firmware or developer path	S
SPIKE Prime	SPIKE app/web app plus official in-app Python help	Scratch-style blocks and Python (MicroPython-based)	**Yes**	Official Python docs expose motor modules with `run`, `run_for_degrees`, `run_to_absolute_position`, and `set_duty_cycle`; Hub OS updates are handled through the app	Official path is app-managed Hub OS updates; I found no official public custom firmware SDK/source in the surveyed docs	
MINDSTORMS EV3	EV3 Programmer/ EV3 Classroom, EV3 MicroPython, MakeCode, Developer Kits	Icon/block programming, Python, MakeCode JavaScript	**Yes**	Strongest official low-level path: communication kits, block dev kit, firmware dev kit, firmware source, direct commands/ bytecodes	Official MicroPython image on microSD; official firmware developer edition and firmware source are available	
Robot Inventor	Retired Robot Inventor app historically; LEGO now points users to the SPIKE app	Scratch-based coding and Python	**Yes**	Good high-level control, but official low-level docs are shallower than EV3; LEGO notes there is no on-hub programming function	Officially app-based; no official public custom firmware path surfaced in this survey	

Platform	Official software surfaces	Official languages	Can you upload custom code?	Low-level motor/sensor control	Official firmware or developer path	S
Powered Up	Powered Up app and official LEGO BLE Wireless Protocol documentation	Block programming in the app; protocol docs enable external implementations	**Yes, but mainly app/ protocol-centric**	Official app blocks include flow, movement, sensor/light logic; official BLE docs describe direct interaction with hubs, motors, and sensors	Official firmware updates occur through apps; no official custom firmware source/ developer edition surfaced in this survey	
BOOST	BOOST app historically; LEGO now routes users to Powered Up app for interactive features	Icon/block-style coding	**Yes, in the app-centric sense**	Official control is comparatively limited and child-oriented; no official Python surfaced in current help pages	Official firmware updates via app; app itself is no longer distributed the way it once was	

A few conclusions follow from that table. **Yes, LEGO kits are programmable and can take custom instructions**—especially SPIKE Prime, EV3, and Robot Inventor. But the **most open official platform is EV3**, not SPIKE Prime, because EV3 still exposes formal developer kits and firmware materials. By contrast, **SPIKE Prime is more current and more educationally polished**, but the official experience is still app-mediated, and LEGO itself describes the hub class as a memory- and processing-constrained microcontroller environment. [22]

That also answers the language question. In the current official LEGO stack, the center of gravity is **blocks/ Scratch-style logic plus Python**. In the materials surveyed for SPIKE Prime, Robot Inventor, BOOST, and Powered Up, I did **not** find Java or C/C++ presented as first-class official programming languages. If Java or C/C++ is a firm requirement, the practical path is to move to **EV3-era developer materials or external controllers such as BrickPi or ev3dev**, not to force modern official LEGO education apps into that role. [23]

Third-party controllers and compatible hardware

If the goal is a robot that can learn, plan, and assemble other robots, **third-party expansion is not just helpful; it is probably necessary**. The reason is that the borderline between “toy robotics” and “robotics

workcell" shows up very quickly once you add cameras, calibration, logs, fleet orchestration, and long-horizon error recovery. [24]

Third-party and compatible options

Option	Type	Languages	What it adds	Main constraints	Why it matters here	Sources
Pybricks	Alternate firmware + cross-hub runtime	Python; Pybricks also advertises block coding	Direct-on-hub execution, unified API across BOOST/ City/Technic/ MINDSTORMS/ SPIKE, and the ability to restore original firmware	Not official LEGO support; firmware install/restore workflow must be managed carefully	Best way to unify the modern LEGO hub families under one programming model	[25]
Raspberry Pi Build HAT	Official Raspberry Pi add-on for LPF2 devices	Python and .NET	Full Linux compute next to LEGO motors/ sensors; camera integration; four LPF2 connectors; RP2040 handles low-level control	Requires Raspberry Pi + external 8V supply for motors; current docs say Raspberry Pi OS Trixie is not yet supported	Easiest route to combine LEGO mechatronics with real computer-vision and planning software	[26]
BrickPi3	Third-party Raspberry Pi bridge board	Python, Scratch, C, Java	4 EV3/NXT motor ports, 4 EV3/NXT sensor ports, Grove I^2C, full Raspberry Pi on top	Uses the older EV3/ NXT connector family, not LPF2; power and battery management are on you	Strong option if you want Raspberry Pi compute plus legacy EV3 hardware and broader language choices	[27]

Option	Type	Languages	What it adds	Main constraints	Why it matters here	Sources
OpenMV Cam RT1062 / H7	Vision coprocessor	MicroPython	On-device AprilTags, QR, blob/feature pipelines, YOLO, UART/ I^2C/SPI/GPIO integration	Separate integration effort; not a LEGO controller by itself	Very good dedicated perception front-end for part localization and visual alignment	28
ev3dev	Linux OS for EV3/ BrickPi-class systems	Many languages; designed for broad Linux stacks	Debian Linux, low-level drivers, access to cameras and USB/ Bluetooth devices	EV3-generation workflow and SD-card image management	Best C/C++-style or Linux-native path if you want a more conventional robotics software environment on LEGO hardware	29
leJOS	Java runtime for EV3	Java	JVM-style programming model and documented robotics API	EV3-only and comparatively mature/older	If Java is mandatory, this is the classic path—not modern SPIKE apps	30

For your specific objective, the most practical split is this:

Best official-first path: SPIKE Prime for LEGO-native modules, but with **external compute added immediately** via Raspberry Pi or workstation. 31

Best unified modern-hub path: Pybricks on SPIKE Prime / Robot Inventor / Powered Up-family hubs, because it makes the runtime more consistent and actually lets programs run directly on the hub with full control of motors and sensors. 32

Best Java/C/C++ path: EV3 + BrickPi/ev3dev/leJOS, because that ecosystem is simply more compatible with low-level and general-purpose programming expectations than current official SPIKE/Powered Up app surfaces. 33

Feasibility of autonomous robot self-assembly

The hard part of your idea is **not** whether LEGO can move motors. The hard part is whether a robot can **reliably align, insert, detect misalignment, recover, and repeat** across many assembly steps. LEGO is

deceptively difficult in this respect because it is a **snap-fit, tolerance-sensitive, contact-rich** domain. Recent research treating LEGO as an assembly testbed is highly relevant here: BrickSim was built specifically because ordinary rigid-body simulation does not faithfully capture snap-fit mechanics; recent tool-tip vision work on LEGO reported that embedded finger vision increased calibration-error tolerance from **0.4 mm up to 2.0 mm**; and recent assembly systems emphasize long-horizon composition, spatial grounding, and online error correction rather than naive pick-and-place alone. [34]

That leads to a very important design rule: **do not target full robot build-out first**. The first autonomous target should be a **restricted module family**. In practice, that means a robot should initially assemble things like: chassis rails, baseplates with known fiducials, two-motor drive modules, sensor towers, or passive structural subassemblies from kitted parts. Once those work, the system can move toward connectorized electrical insertion, pre-fixtured hub mounting, and limited daughter-robot bootstrapping. This is consistent with both classical LEGO planning work that generates plans from enriched 3D models and more recent work on long-horizon LEGO assembly pipelines. [35]

A realistic autonomous assembly cell for LEGO-like robot replication therefore needs more than a gripper. It needs **part presentation**, **fixtures**, **calibration references**, **multi-view or tool-tip cameras**, and ideally some form of **contact supervision**. A particularly strong clue comes from Carnegie Mellon's work on vibro-tactile LEGO assembly, which used contact microphones, force-torque sensors, and cameras as an online self-supervisor; the same thesis also introduced a "BrickPick" end-effector to increase throughput by carrying multiple bricks. That is exactly the kind of practical instrumentation that turns a clever demo into a repeatable cell. [36]

The easiest perception stack is not a giant end-to-end neural system. It is a layered stack: **fiducials/AprilTags and geometric alignment first, learned perception second**. OpenMV explicitly supports AprilTag tracking, QR scanning, and on-device vision/ML in MicroPython, which makes it an appealing low-complexity coprocessor for close-range alignment and tray localization. For larger stacks, a Raspberry Pi with camera and Build HAT or BrickPi class controller gives more room for ROS-style tooling and test infrastructure. [37]

What is mechanically hardest

Assembly function	Difficulty	Why it is hard	Recommended mitigation	Sources
Picking and placing loose bricks/beams	Moderate	Requires reliable part localization and orientation estimation	Kitted trays, overhead vision, restricted part vocabulary, fiducials	[38]
Snap-fit insertion into partial structures	High	Contact-rich, small tolerance margins, occlusion during insertion	Tool-tip camera, force/tactile feedback, compliance, insertion state machine	[39]
Cable routing / battery handling / service tasks	Very high	Not part of LEGO's official coding abstractions; highly irregular and poorly structured	Human-assisted or fixture-assisted stage for early prototypes	[40]

Assembly function	Difficulty	Why it is hard	Recommended mitigation	Sources
Automatic commissioning of a daughter robot	High	Requires firmware management, transport reliability, calibration, and communication setup	USB flashing fixture, scripted tests, known-good port maps, external orchestrator	[41]
Multi-robot parallel assembly	Moderate to high	Safe coordination, collision avoidance, and precedence constraints dominate	Two-layer planning and asynchronous multi-robot execution	[42]

The “learning” part of your concept is therefore best framed as **learning better assembly skills, recovery policies, and instruction generation**, not as expecting a small LEGO hub to train end-to-end from raw vision. Recent LEGO assembly work is much closer to **learning from demonstration, situated guidance, compositional skill chaining, and digital-twin-backed validation** than to unconstrained on-device self-learning. That is the correct model to imitate. [43]

Software factory architecture

A workable software factory for this concept should have a **single source of truth** for each robot design. That source of truth should include the **geometry**, **parts**, **submodels**, **assembly order**, **port map**, **power assumptions**, **calibration references**, and **behavioral code**. BrickLink Studio is especially useful here because it already supports **virtual parts**, **instructions**, **automatic divide-into-steps**, **submodels**, and **parts pricing/availability** linked to the BrickLink catalog. It also exposes a practical data-normalization challenge you should solve early: LEGO uses **Design IDs** and **Element IDs**, while BrickLink/Studio use **Item Numbers**. A software factory needs all of those aligned in one internal schema. [44]

The recommended architecture is:

1. **Design layer**: BrickLink Studio model, normalized part catalog, submodels, and a machine-readable assembly graph.
2. **Instruction layer**: auto-generated build steps, pick lists, and fixture recipes.
3. **Simulation layer**: geometric simulation in Gazebo or Isaac Sim, with snap-fit validation augmented by brick-specific mechanics or empirical tests.
4. **Software layer**: hub firmware/code, Pi/OpenMV code, calibration assets, and interface contracts.
5. **CI/CD layer**: on every design change, rebuild artifacts, run simulation regression tests, verify BOM availability, and publish packages.
6. **Execution layer**: builder cell runs the plan, emits telemetry, and records failures.
7. **Learning layer**: failures become new data for step refinement, grasp tuning, and recovery-policy updates. [45]

Two architectural cautions are especially important.

First, **generic rigid-body simulation is not enough** for this domain. BrickSim’s existence is evidence of that: it was created to model snap-fit connections in interlocking bricks because standard simulators do not capture them well enough. So your pipeline should treat simulation as a layered stack: **ordinary geometry/**

kinematics for planning, **brick-specific mechanics for insertion/disassembly**, and **hardware-in-the-loop** for final acceptance. [46]

Second, the **heavy computation must stay off the hub**. LEGO's own SPIKE Python materials explicitly describe the hub as a microcontroller with limited memory and processing power. That means the hub should own **motor loops, local state, and simple reactive logic**, while external compute should own **perception, planning, dataset management, optimization, and software-factory orchestration**. [47]

A practical CI/CD workflow would therefore look much more like modern software engineering than like traditional toy robotics: the repository should store design files, code, and tests; GitHub Actions or GitLab CI/CD should build and validate artifacts; and every successful build should produce a reproducible bundle containing **instructions, images, port maps, vision models, calibration values, and executable control software** for both the parent builder and the daughter robot. [48]

Cost, safety, and prototype roadmap

Cost and scalability

The economics are good enough for a research prototype and poor for anything resembling industrial replication. Illustrative current prices from primary/vendor pages show why. [49]

Component	Illustrative price/ status	Interpretation	Sources
SPIKE Prime Set	**$429.95**	Strong official base kit, but expensive if you need many ports/actuators across multiple robots	[50]
Powered Up Hub 88009	**$49.99**	Cheap controller node, but only 2 I/O ports and app-oriented workflow	[51]
Technic Hub 88012	**$89.99**	Better than the basic hub for modular daughter robots, but still limited versus external SBC-based control	[52]
Robot Inventor 51515	**$359.99 original LEGO page; retired**	Useful only if already owned; lifecycle risk is material	[53]
MINDSTORMS EV3 31313	**$349.99 original LEGO page; retired**	Legacy but still attractive for openness and developer tooling	[54]
BrickPi	**$259**	Adds Raspberry Pi compute and broader language support, but you still need LEGO motors/sensors/parts	[55]
OpenMV Cam RT1062	**$150**	Good vision coprocessor price point for perception-heavy prototypes	[56]

Component	Illustrative price/ status	Interpretation	Sources
OpenMV Cam H7	**$80; no longer produced**	Useful legacy vision board, but not the long-term choice	[57]

From a scalability perspective, the conclusion is blunt: **LEGO is excellent for modular prototyping, curriculum, and low-volume experimentation**, because the parts are standardized, reusable, and fast to iterate. But **LEGO hubs are expensive per useful compute unit and per industrial-grade assembly capability**. If your goal shifts from “prove the concept” to “scale a robot-building cell,” you will almost certainly want to keep the LEGO geometry and replace more of the electronics stack with **Pi-class SBCs, dedicated motor controllers, or custom boards** while retaining LEGO-compatible structure only where it still buys you speed and modularity. [58]

Safety, legal, and educational constraints

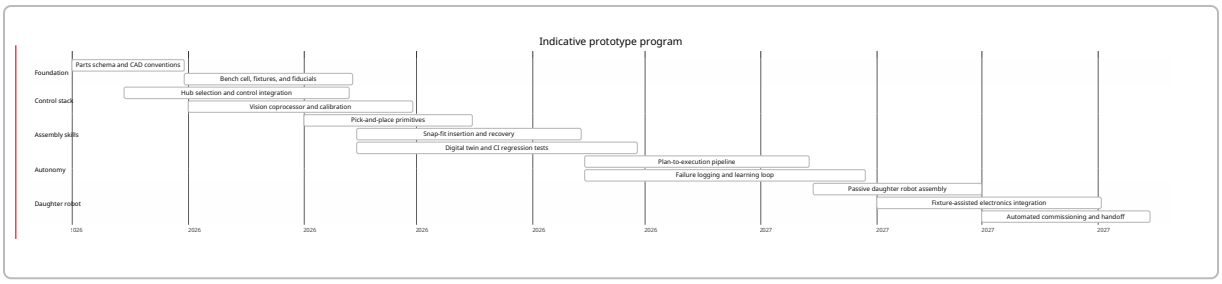
Even if LEGO components are toys or educational products, a self-assembling robot cell should be analyzed as a **robotics workcell**, not as a toy. OSHA’s robotics pages emphasize that many robot accidents occur during **non-routine operation** such as programming, maintenance, testing, setup, or adjustment, and OSHA notes there is no single robotics-specific OSHA standard—so the burden shifts to machine guarding, lockout/tagout, and general machinery safeguards. That matters here because prototyping a builder robot naturally involves repeated calibration, setup, manual intervention, and testing inside the work envelope. [59]

If this system is used in schools or youth programs, cameras and cloud-connected coding environments create data-governance questions. The U.S. Department of Education states that FERPA gives parents or eligible students rights over education records and disclosure of personally identifiable information in those records. The FTC’s COPPA guidance states that the rule applies to online services, apps, and IoT devices directed to children under 13 that collect personal information, including photos, video, audio, persistent identifiers, and geolocation. In practical terms, if you log student video or voice, sync accounts, or store individualized performance traces in a school deployment, you need deliberate privacy design—not just a robotics lesson plan. [60]

Educationally, LEGO still has a major advantage: the platform is understandable, modular, and motivating, and Raspberry Pi explicitly positions Build HAT as a bridge between LEGO Technic components and full Raspberry Pi programming. That makes the concept unusually good for **teaching software-factory principles, versioned robotics, digital twins, and embodied AI**—provided the project is scoped as a staged research platform rather than sold as autonomous self-replication on day one. [61]

Recommended prototype roadmap

The most credible roadmap is a staged one: **do not try to “build robots that build robots” in a single jump**. Start with a one-cell builder that assembles passive modules from kitted trays; then add perception, insertion supervision, and digital twins; then progress to fixture-assisted daughter-robot completion and automated commissioning. That ordering is consistent with the official compute constraints of LEGO hubs, the capabilities of Pi/OpenMV expansions, and the current state of LEGO assembly research. [62]



A good milestone structure is:

Milestone	Deliverable	Exit criterion
Prototype cell	One builder robot cell with controlled trays and fiducials	Repeats passive subassembly builds with acceptable success rate across many trials
Controlled insertion	Snap-fit insertion with tool-tip vision and/or tactile supervision	Can detect and recover from common insertion failures without manual reset
Software factory	Automated generation of BOM, steps, simulation artifacts, and code bundles	A design change automatically regenerates artifacts and passes regression checks
Daughter robot build	Robot can assemble a restricted daughter platform	Finished module passes power-on, motion, and communication tests
Learning loop	Recorded failures feed policy/step refinement	Measured improvement in cycle time or success rate across versions

My recommendation is to make the **first daughter robot deliberately simple**: one standardized drive base, one fixed sensor mast, one known hub/controller family, one known battery/power path, and no loose cable routing. Once that works, you can expand the robot family through **configurable top modules** rather than through unrestricted whole-robot variation. That is exactly the kind of product-line discipline that software factories are good at. [63]

Open questions and limitations

Official LEGO documentation does **not prominently publish exact CPU and RAM figures** for several modern hubs, so some embedded resource analysis for SPIKE Prime, Robot Inventor, BOOST, and current Powered Up hubs remains less precise than for EV3, Build HAT, or OpenMV. LEGO's own language emphasizes the practical point anyway: these hubs are constrained microcontroller environments. [64]

Support lifecycle is also uneven. Robot Inventor, EV3, and BOOST are retired; SPIKE Prime is still sold in some markets, but at least one regional LEGO Education page now flags retirement on June 30, 2026. If you want a platform that will still be serviceable after a multi-semester prototype, lifecycle risk should be treated as a design parameter, not an afterthought. [65]

Finally, some of the most relevant evidence on LEGO self-assembly comes from **recent theses and preprints**, which are valuable primary sources but not the same thing as long-running industrial validation.

The right conclusion is not “this is proven at production scale”; it is “this is credible enough to justify a serious prototype, provided the scope is constrained and the architecture is hybrid.” [66]

---

[1] [7] [31] [41] Product Info
https://education.lego.com/en-us/teacher-resources/lego-education-spike-prime/support-technical-info/lego-education-spike-prime-support-technical-info-product-info?utm_source=chatgpt.com

[2] [18] [22] MINDSTORMS EV3 Support | Everything You Need | LEGO® Education
https://education.lego.com/en-us/product-resources/mindstorms-ev3/teacher-resources/python-for-ev3/?utm_source=chatgpt.com

[3] [24] [26] [58] [61] Build HAT - Raspberry Pi Documentation
https://www.raspberrypi.com/documentation/accessories/build-hat.html?utm_source=chatgpt.com

[4] [6] [43] Robotic LEGO Assembly and Disassembly from Human Demonstration
https://arxiv.org/abs/2305.15667?utm_source=chatgpt.com

[5] SPIKE Prime Support | Everything You Need | LEGO® Education
https://education.lego.com/en-us/product-resources/spike-prime/downloads/python/?utm_source=chatgpt.com

[8] [16] [23] [47] [62] [64] LEGO Education SPIKE
https://spike.legoeducation.com/prime/modal/help/lls-help-python?utm_source=chatgpt.com

[9] [48] GitHub Actions · GitHub
https://github.com/features/actions?utm_source=chatgpt.com

[10] [11] [49] [50] SPIKE™ Prime – STEAM Set - Grades 6 - 8 | LEGO® Education
https://education.lego.com/en-us/products/lego-education-spike-prime-set/45678?utm_source=chatgpt.com

[12] Help Topics
https://www.lego.com/en-us/service/help-topics/article/lego-mindstorms-ev3-troubleshooting-help?utm_source=chatgpt.com

[13] [19] [65] Help Topics
https://www.lego.com/en-us/service/help-topics/article/About-MINDSTORMS-Robot-Inventor?utm_source=chatgpt.com

[14] Hub 88009 | Powered UP | Buy online at the Official LEGO® Shop US
https://www.lego.com/en-us/product/88009?utm_source=chatgpt.com

[15] [21] Help Topics
https://www.lego.com/en-us/service/help-topics/article/All-About-LEGO-Boost?utm_source=chatgpt.com

[17] SPIKE Prime | Student App Download | LEGO® Education
https://education.lego.com/en-us/downloads/spike-app/software/?utm_source=chatgpt.com

[20] Help Topics
https://www.lego.com/en-us/service/help-topics/article/lego-powered-up-programming-blocks?opt-out-modal=show&utm_source=chatgpt.com

[25] [32] Pybricks Documentation — pybricks v3.6.1 documentation
https://docs.pybricks.com/?utm_source=chatgpt.com

[27] [33] BrickPi3 Technical Details - Behavior, Design, and Electrical Schematic of the BrickPi3
https://www.dexterindustries.com/BrickPi/brickpi3-technical-design-details/?utm_source=chatgpt.com

28 37 38 OpenMV MicroPython 1.28 documentation
https://docs.openmv.io/?utm_source=chatgpt.com

29 ev3dev Home
https://www.ev3dev.org/?utm_source=chatgpt.com

30 LeJOS, Java for Lego Mindstorms / EV3
https://lejos.sourceforge.io/ev3.php?utm_source=chatgpt.com

34 46 BrickSim: A Physics-Based Simulator for Manipulating Interlocking Brick Assemblies | Cool Papers - Immersive Paper Discovery
https://papers.cool/arxiv/2603.16853?utm_source=chatgpt.com

35 42 63 (PDF) LegoBot: Automated Planning for Coordinated Multi-Robot Assembly of LEGO structures*
https://www.researchgate.net/publication/349648164_LegoBot_Automated_Planning_for_Coordinated_Multi-Robot_Assembly_of_LEGO_structures?utm_source=chatgpt.com

36 66 Improving Robotic Lego Assembly with Vibro-Tactile Feedback - Robotics Institute Carnegie Mellon University
https://publications.ri.cmu.edu/improving-robotic-lego-assembly-with-vibro-tactile-feedback?utm_source=chatgpt.com

39 Eye-in-Finger: Smart Fingers for Delicate Assembly and Disassembly of LEGO
https://arxiv.org/abs/2503.06848?utm_source=chatgpt.com

40 Help Topics
https://www.lego.com/en-us/service/help-topics/article/about-the-lego-mindstorms-robot-inventor-app?utm_source=chatgpt.com

44 Introduction to Studio – Studio Help Center
https://studiohelp.bricklink.com/hc/en-us/articles/5404381697559-Introduction-to-Studio?utm_source=chatgpt.com

45 Isaac Sim - Robotics Simulation and Synthetic Data Generation | NVIDIA Developer
https://developer.nvidia.com/isaac/sim?utm_source=chatgpt.com

51 Hub 88009 | Powered UP | Buy online at the Official LEGO® Shop US
https://www.lego.com/en-us/product/hub-88009?utm_source=chatgpt.com

52 Technic™ Hub 88012 | Technic™ | Buy online at the Official LEGO® Shop US
https://www.lego.com/en-us/product/88012?utm_source=chatgpt.com

53 Robot Inventor 51515 | MINDSTORMS® | Buy online at the Official LEGO® Shop US
https://www.lego.com/en-us/service/help/mindstorms_robot_inventor/mindstorms_robot_inventor/what-comes-in-the-lego-mindstorms-robot-inventor-51515-kA009000001dcjlCAA?utm_source=chatgpt.com

54 LEGO® MINDSTORMS® EV3 31313 | MINDSTORMS® | Buy online at the Official LEGO® Shop US
https://www.lego.com/en-us/product/lego-mindstorms-ev3-31313?utm_source=chatgpt.com

55 BrickPi - Dexter Industries
https://www.dexterindustries.com/brickpi-2020/?utm_source=chatgpt.com

56 Small - Affordable - Expandable – OpenMV
https://openmv.io/?utm_source=chatgpt.com

57 OpenMV Cam H7
https://openmv.io/products/openmv-cam-h7?utm_source=chatgpt.com

59 Robotics - Overview | Occupational Safety and Health Administration
https://www.osha.gov/robotics?utm_source=chatgpt.com

60 Frequently Asked Questions | U.S. Department of Education
https://www.ed.gov/about/contact-us/faqs/Student%20Records%20and%20Privacy?utm_source=chatgpt.com